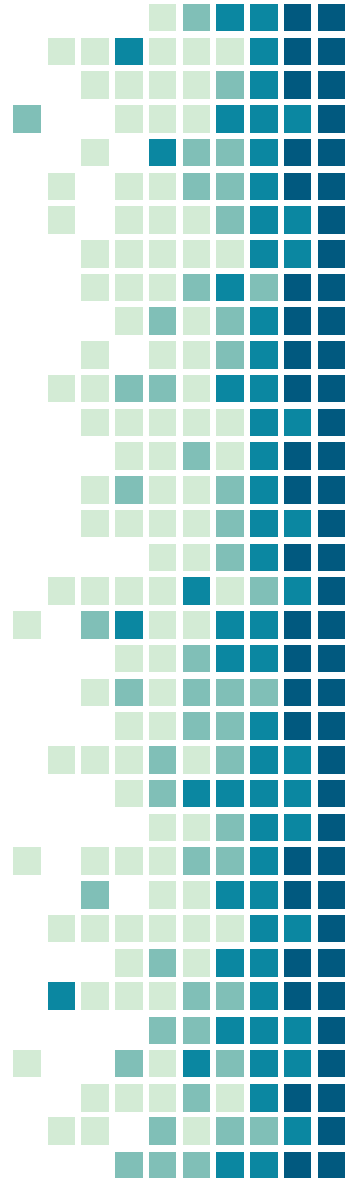




What is MVC Pattern

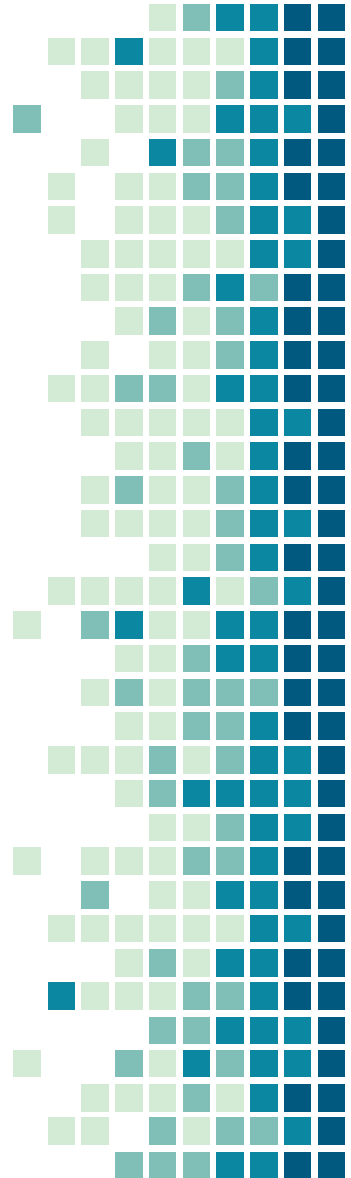
Today's Learning.

- What is Design Pattern?
- What is MVC Design Pattern?
- Why MVC Design Pattern?
- What is Controller & Actions ?
- What is Routing in MVC?
- Basic Working Flow ASP.NET MVC.



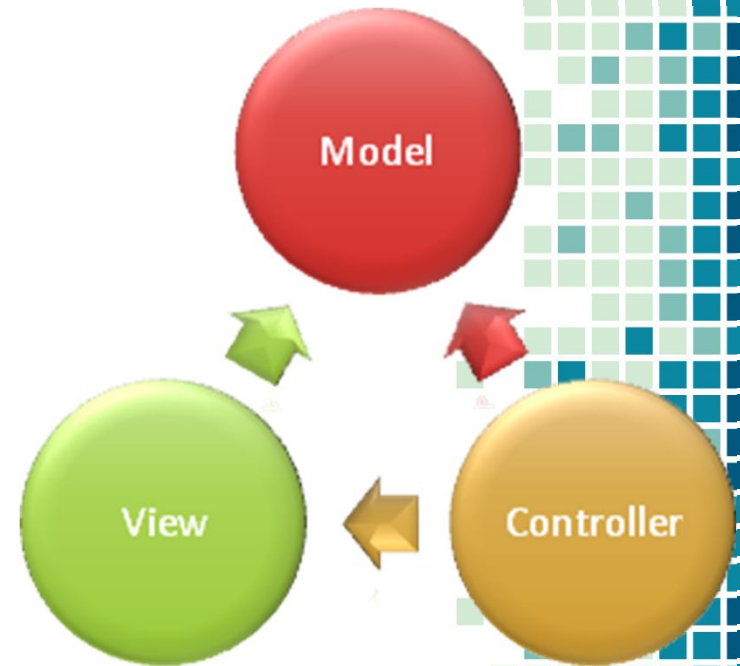
What is Design Pattern?

- Design patterns tell you how to write a code.
- Design patterns provide you some set of guidelines, rules, specifications that need to follow while writing code.
- Every Design Pattern has its own Advantages.
- Design patterns are always implemented by developer or programmer.
- Few popular design patterns are – MVC , MVP, MVVM, Repository etc.



What is MVC Design Pattern?

- MVC stands for **Model**, **View**, and **Controller**.
- It is an design pattern used to organize and structure your application code.
- MVC helps achieve separation of concerns, making it easier to develop, maintain and test the application.



Why MVC Design Pattern?

- MVC has only principal SOC (Separation of Concerns).
- SOC say's find out the basic concerns of any application and separate those concerns from each other so that load carrying capability of application is going to grow (i.e. your application can handle more number of users).
- MVC is a design pattern used to decouple/ separate User-Interface (UI), Data and Application Logic.



Why MVC Design Pattern?

- If you look at to any program it will have following three things –
 - Data
 - User Interface
 - Logic
- MVC Says separate Data , User Interface and Logic from each other so that load carrying capability of application is going to grow



What is Model?

Model:-

- M stands for Model. Model is part of MVC is responsible for the application's data logic.
- Model is responsible for storing and retrieving data from back-end data stores.
- The Model may also include mechanisms for validating data and carrying out other data-related tasks.
- Indirectly Model is responsible for all Data Related Activities.



What is View?

View:-

- V stands for View. View is responsible for UI (User Interface) part of Application.
- View provides the UI necessary to interact with the Application.
- It includes components needed to display the data and enables users to interact with that data.
- For example, the view layer might include buttons, links, tables, drop-down lists or text boxes.

What is Controllers?

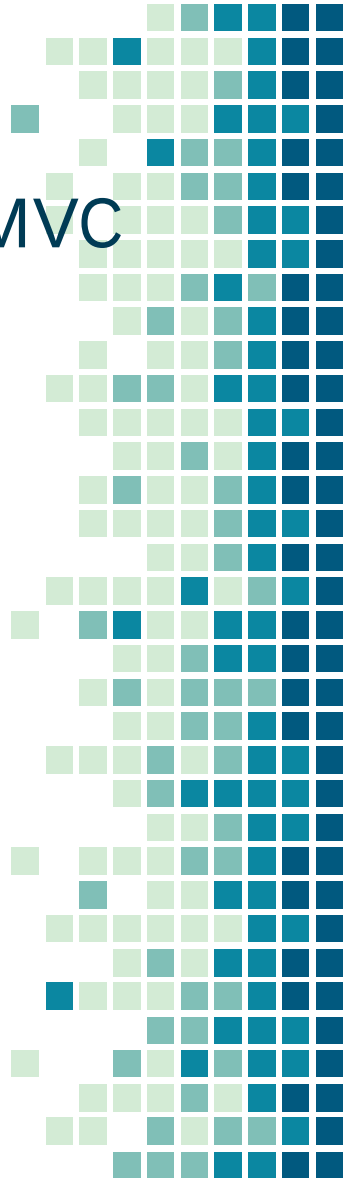
Controllers:-

- C Stands for Controller. The controller is responsible for application logic.
- Controller contains a code that facilitate communications across the application.
- Controller act as an interface between the view and model.
- The controller is sometimes viewed as the brains of the application, keeping everything moving and in sync.



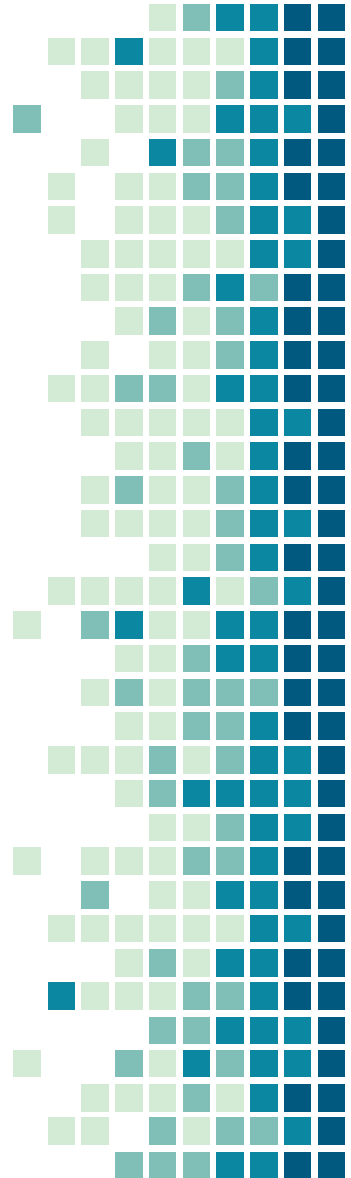
Few Web Application Frameworks.

- Few web application frameworks those follows MVC design pattern –
 - ❖ ASP.NET core
 - ❖ ASP.NET MVC
 - ❖ Ruby on Rails
 - ❖ Struts
 - ❖ Larval
 - ❖ PHP Zend etc.



Controllers in Details

- A controller is responsible for controlling the way that a user interacts with an MVC application.
- A controller contains the flow control logic for an ASP.NET MVC application.
- A controller determines what response to send back to a user when a user makes a browser request.



Controllers in Details

- Controller is just a class located in Controllers folder will have a suffix Controller.
- In case of ASP.NET Core Controller Class can be POCO.
- Controller class can be inherited from built in class called Controller to consume View Benefits.
- Contains set of methods called actions and actions will have one to one mapping between user interaction and action.



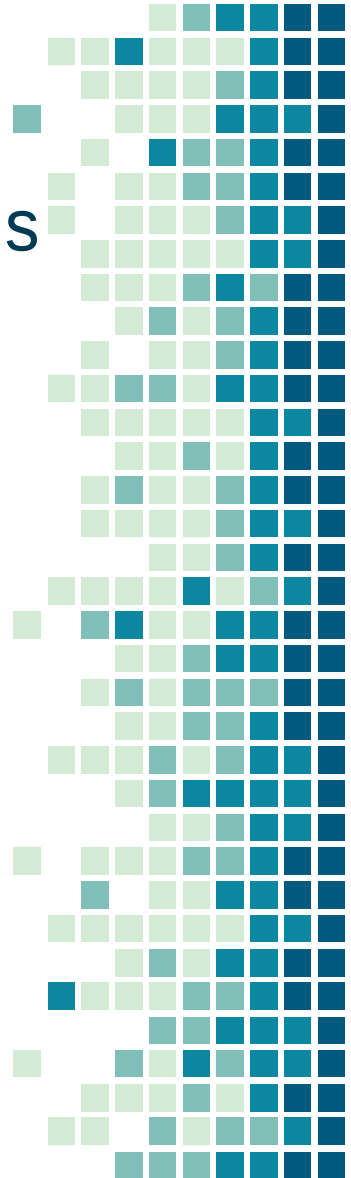
Controllers in Details

- Controller is just a class located in Controllers folder will have a suffix Controller.
- In case of ASP.NET Core Controller Class can be POCO.
- Controller class can be inherited from built in class called Controller to consume View Benefits.
- Controller class should be public can not be private, protected, abstract, static or sealed.
- Contains set of methods called Actions.



What is Action?

- Any public method defined inside a controller is called action.
- Actions will have one to one mapping between user interaction and action.
- The action method must be public.
- The method cannot be a static method.
- The method cannot be an extension method.
- The method cannot be an abstract method.



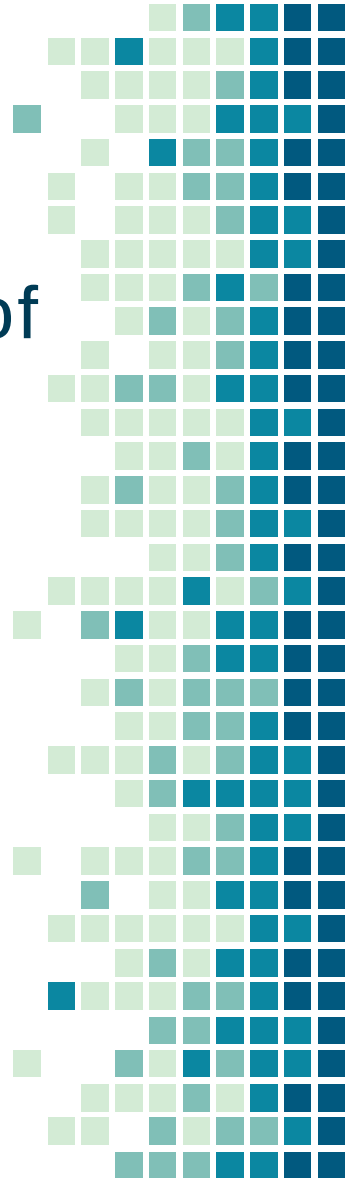
What is Routing?

- Routing is process by which we can map user request to specific action of specific controller.
- ASP.NET MVC supports URL Routing.
- Route Engine manage the Routing.
- Route Engine Resolves Request URL and decides which action of which controller to be called.
- **Route Table** – contains route map entries (records) which contains URL patterns contains information of action and controller.



What is Routing?

- Route table also contains a default route that will be get added to route table at the startup of the application.



Basic Working Flow



2. Route to appropriate Laravel Controller

1. Submit User Request



Routing

Controller

3. Interact with Data Model

4. Controller invokes results View

View

Model

5. Render view in users browser



Database



Use MVC in ASP.NET Core.

- Since ASP.NET Core is modular, we need to implement following two steps to use MVC –
 1. Register the MVC Service to IOC container using Configure Services methods of Startup Class.
 2. Register Routing Middleware in Request Pipe Line.



Register MVC service to IOC.

- To register MVC service to IOC asp.net core provides you two extension methods –
 1. **AddMvcCore():** - This method adds only core mvc services to IOC Container.
 2. **AddMvc():** - This method will register full fletch MVC Service to IOC Container, internally calls AddMvcCore().



Inject MVC Middleware.

- ASP.NET Core provides you a built in middleware called Mvc you can inject it request pipeline to handle routing process for MVC Application.
- Using this middleware you can register the URL patterns to Route Table those will be followed by route engine to make decision which action of which controller will be called.
- To inject MVC middleware use UseMvc() extension method.

Inject MVC Middleware.

```
App.UseMvc(routes=> {  
    routes.MapRoute("default", "{controller=Home}/{action=index}/{id?}")  
})
```

- This will get you error ...

How to Solve IT?

- ASP.NET Core 3.1 supports endpoint routing and by default endpoint routing middleware do not use the `UseMvc()` / `UseMvcWithDefaultRoute()` middleware.
- It uses different service and middle ware that we will see.
- So we need to disable the endpoint routing by padding `enableendpointrouting=false`.
- `services.AddMvcCore(p=>p.EnableEndpointRouting=false);`

THANKS!

Any questions?

You can find me at:

www.ritechpune.com

contact@ritechpune.com

9730010404

